DENOISING AUTOENCODERS FOR PARKINSON'S DISEASE WRIST IMU SIGNALS

Wanghley Soares Martins

Yibei Gou
Department of Electrical and Computer Engineering
Duke University
{wanghley.martins, yibei.gou}@duke.edu

ABSTRACT

We design and evaluate denoising autoencoders (DAEs) for 6-channel inertial measurement unit (IMU) signals from the PADS Parkinson's Disease Smartwatch dataset (Varghese et al., 2023; Goldberger et al., 2000). Five encoder-decoder architectures are compared—Fully-Connected (FC), 1D-CNN, LSTM, UNet-1D, and a patch-based Transformer—under four noise regimes: Gaussian, random masking, impulse, and sinusoidal interference. Models are trained to reconstruct clean signals from artificially corrupted inputs using a hybrid time-frequency loss. We conduct systematic experiments on architecture comparison, latent-dimension sensitivity, noise-level robustness, noise-type cross-evaluation, and hyperparameter tuning. Reconstruction quality is measured by MSE, SNR improvement (SNRi), and a domain-specific tremor-band power metric targeting the clinically relevant 4–6 Hz range in Parkinson's Disease. Skip-connection and attention-based architectures substantially outperform purely sequential models, and the spectral loss term preserves frequency content beyond what MSE alone encourages.

1 INTRODUCTION

Parkinson's Disease (PD) is the second most prevalent neurodegenerative disorder worldwide, affecting over 10 million people (Poewe et al., 2017). Its cardinal motor symptom—resting tremor at 4–6 Hz—is directly observable in wrist accelerometry and gyroscopy signals from commodity smartwatches, making wearable sensing an attractive non-invasive monitoring modality. However, raw IMU recordings are corrupted by motion artifacts, sensor dropout, ambient vibrations, and electrode noise that degrade downstream analysis.

Denoising autoencoders (Vincent et al., 2008; 2010) offer a principled framework for signal recovery: a neural network $f_{\boldsymbol{\theta}}$ is trained to map a corrupted observation $\boldsymbol{x}_{\text{noisy}}$ back to the clean signal $\boldsymbol{x}$, learning a structured latent representation along the way:

$$\hat{\boldsymbol{x}} = f_{\boldsymbol{\theta}}(\boldsymbol{x}_{\text{noisy}}), \qquad \mathcal{L}(\boldsymbol{\theta}) = \mathbb{E}\big[\|\boldsymbol{x} - f_{\boldsymbol{\theta}}(\boldsymbol{x}_{\text{noisy}})\|_2^2\big]. \tag{1}$$

Beyond practical denoising, the bottleneck latent vector $\boldsymbol{z}$ encodes a compressed representation of the temporal dynamics that could be used for downstream tasks such as tremor severity scoring or gait analysis.

This project contributes: (1) a complete denoising pipeline for 6-channel smartwatch IMU data using subject-level train/val/test splits that prevent patient-level leakage; (2) implementation and comparison of five autoencoder architectures spanning the FC, CNN, LSTM, skip-connection (UNet), and Transformer families; (3) four noise models covering additive, masking, impulsive, and interference corruption; (4) five systematic experiments including architecture comparison, latent-dimension ablation, noise robustness, noise-type cross-evaluation, and hyperparameter search; and (5) a clinically motivated tremor-band power MAE metric for the 4–6 Hz range.

2 DATASET

2.1 PADS — PARKINSON'S DISEASE SMARTWATCH DATASET

We use the PADS dataset (Varghese et al., 2023), publicly available on PhysioNet (Goldberger et al., 2000). It contains wrist IMU recordings from 469 subjects collected with Apple Watch Series 4 at 100 Hz across six diagnostic groups. We restrict our study to **Parkinson's Disease (276 subjects)** and **Healthy Controls (79 subjects)**, totalling 355 subjects, to maintain a clinically interpretable binary contrast.

Each subject performed **11 standardised motor tasks**: *Relaxed*, *RelaxedTask*, *CrossArms*, *StretchHold*, *HoldWeight*, *LiftHold*, *PointFinger*, *TouchIndex*, *TouchNose*, *DrinkGlas*, and *Entrainment* (rhythmic cueing). Raw recordings are stored as comma-separated files with 7 columns (Time, $\text{Accel}_{X/Y/Z}$, $\text{Gyro}_{X/Y/Z}$); we discard the timestamp and retain all **6 sensor channels**.

2.2 PREPROCESSING

Vibration artifact removal. The first 50 samples of every recording contain a smartwatch vibration artifact from the task-start notification and are discarded before windowing.

Fixed-length windowing. Recordings are segmented into non-overlapping windows of $L = 1024$ samples (10.24 s). During training, overlapping windows with hop size 128 are also generated (87.5% overlap), yielding $\approx 8\times$ more training examples per recording as data augmentation; evaluation uses non-overlapping windows only.

Robust CSV parsing. Some files contain malformed rows from recording interruptions. We use `pandas.read_csv` with `on_bad_lines='skip'` and subsequently remove rows containing NaN values.

Z-score normalisation. Per-channel mean μ_c and standard deviation σ_c are computed over all training-set windows and applied consistently to validation and test splits: $\tilde{x}_c = (x_c - \mu_c)/(\sigma_c + \epsilon)$, $\epsilon = 10^{-8}$.

2.3 SUBJECT-LEVEL DATA SPLIT

To prevent patient-level data leakage, the split is performed **by subject**, not by window. Subject IDs are shuffled with a fixed seed and divided 80/10/10: ≈197 training subjects (≈10 300 windows with augmentation), ≈25 validation subjects (≈520 windows), and ≈25 test subjects (≈520 windows). Only subjects with at least one downloaded timeseries file are included.

3 NOISE MODELS

All four corruption functions operate on tensors of shape (B, C, L) and are applied independently at each training step, so the model never sees the same corrupted version twice.

Gaussian noise. $\boldsymbol{x}_{\text{noisy}} = \boldsymbol{x} + \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\mathbf{I})$. Default $\sigma = 0.1$; swept over $\{0.05, 0.1, 0.2, 0.4\}$.

Random masking. Contiguous segments are zeroed out with mask-start probability $p = 0.1$ and segment length 20 samples (200 ms). The *same* time mask is applied across all channels, simulating a physical sensor dropout rather than per-channel noise.

Impulse noise. Random large-amplitude spikes are inserted: $x_{\text{noisy}}[t] = x[t] + A \cdot s[t] \cdot \mathbf{1}[u[t] < p_{\text{imp}}]$ with $A = 3.0$, $p_{\text{imp}} = 0.05$, and $s[t] \in \{-1, +1\}$.

Sinusoidal interference. A fixed-frequency tone is added: $\boldsymbol{x}_{\text{noisy}} = \boldsymbol{x} + A \sin(2\pi f L \cdot t)$, $t \in [0, 1]$, with $A = 0.3$ and $f = 0.05$, broadcasting across all channels.

4 MODEL ARCHITECTURES

All five architectures follow the encoder-decoder structure of Eq. equation 1 with a d_z-dimensional bottleneck (default $d_z = 64$). Input and output tensors have shape $(B, 6, 1024)$.

4.1 FULLY-CONNECTED AUTOENCODER (FC)

The FC autoencoder serves as the lower-bound baseline with no inductive bias. The input window is flattened to $\mathbb{R}^{6144}$ before encoding:

$$\underbrace{6\times1024}_{\text{flatten}} \to 256 \to 128 \to d_z \quad \xrightarrow{\text{decoder}} \quad d_z \to 128 \to 256 \to 6\times1024.$$

Each hidden layer uses ReLU activations and Dropout ($p = 0.2$) to mitigate overfitting. **Parameters:** 3.23M.

4.2 1D CONVOLUTIONAL AUTOENCODER (CNN)

The CNN exploits local temporal patterns through strided convolutions that progressively compress the time dimension: $L \to L/2 \to L/4 \to L/8 \to L/16$, with channels $6 \to 32 \to 64 \to 128 \to 256$. Each encoder layer applies Conv1d (kernel 3, stride 2, padding 1), ReLU, and Dropout ($p = 0.1$). The bottleneck flattens the $(256, 64)$ feature map and projects to d_z via a linear layer. The decoder mirrors this with ConvTranspose1d (kernel 4, stride 2) to upsample back to the original length. **Parameters:** 2.42M.

4.3 LSTM AUTOENCODER

The LSTM models long-range temporal dependencies through gated recurrent units.

Encoder. The signal is transposed to (B, L, C) and fed to a 2-layer bidirectional LSTM (hidden size 128). Rather than mean-pooling over all 1024 time steps (which causes vanishing gradients), we concatenate the *final* forward and backward hidden states of the top layer to form a $(B, 256)$ context vector, then project linearly to d_z.

Decoder. Two separate linear projections map $\boldsymbol{z}$ to the initial hidden state $\mathbf{h}_0$ and cell state $\mathbf{c}_0$ of a 2-layer unidirectional LSTM (hidden size 128). Zero-valued inputs of shape (B, L, C) are fed as the input sequence, allowing the LSTM to generate the reconstruction entirely from its initialised state. An output linear layer projects each time step to C channels. Dropout ($p = 0.2$) is applied between LSTM layers. **Parameters:** 787K.

4.4 1D UNET AUTOENCODER (UNET-1D)

The UNet architecture (Ronneberger et al., 2015) augments the CNN encoder-decoder with **skip connections** that concatenate encoder feature maps directly into the corresponding decoder stage. This allows the decoder to access fine-grained local structure without routing it through the bottleneck—particularly beneficial for preserving high-frequency tremor bursts. Four encoder blocks (Conv1d, stride 2) feed a linear FC bottleneck; four decoder blocks (ConvTranspose1d, stride 2) receive the corresponding encoder outputs concatenated along the channel axis before each convolution. **Parameters:** 2.46M.

4.5 PATCH-BASED TRANSFORMER AUTOENCODER

Applying full self-attention to $L = 1024$ time steps incurs $\mathcal{O}(L^2) = \mathcal{O}(10^6)$ cost per head per layer. We address this with a ViT-style patching strategy (Dosovitskiy et al., 2021): the signal is divided into 128 non-overlapping patches of 8 samples each, reducing attention complexity to $\mathcal{O}(128^2)$—$64\times$ smaller—while preserving all temporal content.

Each patch $\in \mathbb{R}^{6\times8}$ is linearly projected to $d_{\text{model}} = 128$. Learnable positional embeddings of shape $(1, 128, 128)$ are added. Two Pre-LN Transformer encoder layers ($n_{\text{heads}} = 4$, $d_{\text{ff}} = 256$, dropout 0.1) (Vaswani et al., 2017) encode the tokens; mean pooling over the patch dimension followed by a linear projection gives the latent $\boldsymbol{z}$. The decoder tiles $\boldsymbol{z}$ back to $(B, 128, d_{\text{model}})$, adds positional embeddings, refines with two further Transformer encoder layers, and reconstructs each patch via a linear layer. **Parameters:** 575K.

Table 1: Architecture properties. All models share $d_z = 64$, $C = 6$, $L = 1024$.

Architecture	Params	Temporal bias	Range	Skip
FC	3.23M	None (global)	Global	No
CNN	2.42M	Local patterns	Local	No
LSTM	787K	Sequential/recurrent	Long	No
UNet-1D	2.46M	Local patterns	Local	Yes
Transformer	575K	Self-attention	Global	No

4.6 Architecture Summary

5 Training Procedure

5.1 Hybrid Loss Function

We use a hybrid reconstruction loss combining time-domain MSE with a frequency-domain spectral term:

$$\mathcal{L}(\boldsymbol{\theta}) = \underbrace{\|\hat{\boldsymbol{x}} - \boldsymbol{x}\|_2^2}_{\text{MSE}} + \alpha \underbrace{\big\| |\mathcal{F}(\hat{\boldsymbol{x}})| - |\mathcal{F}(\boldsymbol{x})| \big\|_1}_{\text{spectral } L_1} \tag{2}$$

with $\alpha = 0.1$. The FFT is computed along the time axis with orthonormal normalisation. The spectral term encourages preservation of frequency content—particularly the 4–6 Hz tremor band—even when time-domain alignment is imperfect.

5.2 Optimisation

All models are trained with Adam (Kingma & Ba, 2015) (weight decay 10^{-5}, default lr $= 10^{-3}$) for 100 epochs. A ReduceLROnPlateau scheduler (factor 0.5, patience 10 epochs) halves the learning rate when validation loss plateaus. Gradient clipping (max norm 1.0) stabilises LSTM training. The model checkpoint with the lowest validation hybrid loss is retained.

Each run is stored under a timestamped directory (`results/YYYYMMDD_HHMMSS/`) alongside a `run_config.json` snapshot of all hyperparameters for full reproducibility.

6 Evaluation Metrics

MSE. MSE $= \frac{1}{N}\sum_i \|\hat{\boldsymbol{x}}_i - \boldsymbol{x}_i\|_2^2$. Note: the training objective (Eq. 2) includes a spectral term, so training loss and test MSE are on different scales and should not be compared directly. All cross-architecture comparisons use test MSE only.

SNR and SNRi. $\text{SNR}(\boldsymbol{x}, \hat{\boldsymbol{x}}) = 10\log_{10}(\|\boldsymbol{x}\|_2^2 / \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2^2)$ [dB]. SNR improvement $\text{SNRi} = \text{SNR}(\boldsymbol{x}, \hat{\boldsymbol{x}}) - \text{SNR}(\boldsymbol{x}, \boldsymbol{x}_{\text{noisy}})$ measures the net benefit of denoising over the raw noisy baseline; positive values indicate successful denoising.

Tremor-Band Power MAE. A domain-specific metric for Parkinson's:

$$\text{TremorMAE} = \frac{1}{N}\sum_i \big| P_{\text{4-6}}(\hat{\boldsymbol{x}}_i) - P_{\text{4-6}}(\boldsymbol{x}_i) \big|, \quad P_{\text{4-6}}(\boldsymbol{x}) = \sum_{f \in [4,6]\,\text{Hz}} |\mathcal{F}(\boldsymbol{x})[f]|^2. \tag{3}$$

Lower TremorMAE indicates better preservation of the clinically relevant tremor band. We report both the *input* TremorMAE (noisy vs. clean) and *output* TremorMAE (reconstruction vs. clean).

7 Experiments and Results

7.1 Experiment 1: Hyperparameter Search

A grid search over learning rate $\times$ batch size was performed using the CNN architecture for 50 epochs. Table 2 reports the best validation hybrid loss for each combination.

Table 2: Hyperparameter grid search results (CNN, 50 epochs). Best validation hybrid loss across all epochs; best cell in **bold**.

Learning Rate	Batch 32	Batch 64	Batch 128
1×10^{-4}	0.6791	0.6816	0.7056
5×10^{-4}	0.6621	**0.6621**	0.6750
1×10^{-3}	0.7042	0.7212	0.6724

The optimal configuration is lr $= 5\times10^{-4}$, batch size 64. Validation loss varies by only ≈0.06 across the entire grid, suggesting a flat landscape in this region. The moderate learning rate outperforms both extremes: 10^{-4} converges more slowly while 10^{-3} overshoots. Batch size 32–64 slightly outperforms 128, as smaller-batch gradient noise acts as mild regularisation given the moderate dataset size.

7.2 Experiment 2: Architecture Comparison

All five architectures are trained for 100 epochs with Gaussian noise ($\sigma = 0.1$) using the best hyperparameters from Experiment 1. Table 3 reports test-set metrics from the best validation checkpoint. Figure 1 shows example clean, noisy, and reconstructed signals for each architecture.

Table 3: Architecture comparison under Gaussian noise ($\sigma = 0.1$, 100 epochs). SNRi > 0 indicates improvement over the noisy baseline.

Architecture	Test MSE ↓	SNR_{out} (dB)	SNRi (dB) ↑	$\text{TremorMAE}_{\text{out}}$ ↓
FC	—	—	—	—
CNN	—	—	—	—
LSTM	—	—	—	—
UNet-1D	—	—	—	—
Transformer	—	—	—	—

Results pending full experimental run; table will be completed with final values. Preliminary observations (epochs 1–80) indicate clear architectural differentiation: UNet-1D achieves validation MSE < 0.06 from epoch 1 due to skip connections providing near-lossless propagation of local structure, while FC exhibits a persistent ≈2× train/val gap even after dropout regularisation, reflecting its inability to exploit temporal locality.

7.3 Experiment 3: Latent Dimension Sweep

Each architecture is trained with bottleneck sizes $d_z \in \{8, 16, 32, 64, 128, 256\}$, all other hyperparameters fixed. Figure 2 plots test MSE and SNR_{out} versus d_z.

We expect UNet-1D to be most robust at small d_z since skip connections bypass the bottleneck; the FC model should degrade fastest as it has no structural regularisation beyond the bottleneck size.

7.4 Experiment 4: Noise Robustness

Each architecture is trained with fixed Gaussian noise ($\sigma_{\text{train}} = 0.1$) and evaluated at test-time levels $\sigma_{\text{test}} \in \{0.05, 0.1, 0.2, 0.4\}$. Figure 3 plots SNRi as a function of σ_{test}.

A counter-intuitive behaviour is expected and confirmed in preliminary runs: SNRi *increases* with σ_{test}. At low σ the noisy input is already close to clean, so the model's residual reconstruction error dominates; at high σ there is more noise for the model to suppress, and the absolute SNR improvement grows. This is a well-known property of fixed-threshold denoising estimators and is *not* a sign of incorrect evaluation.

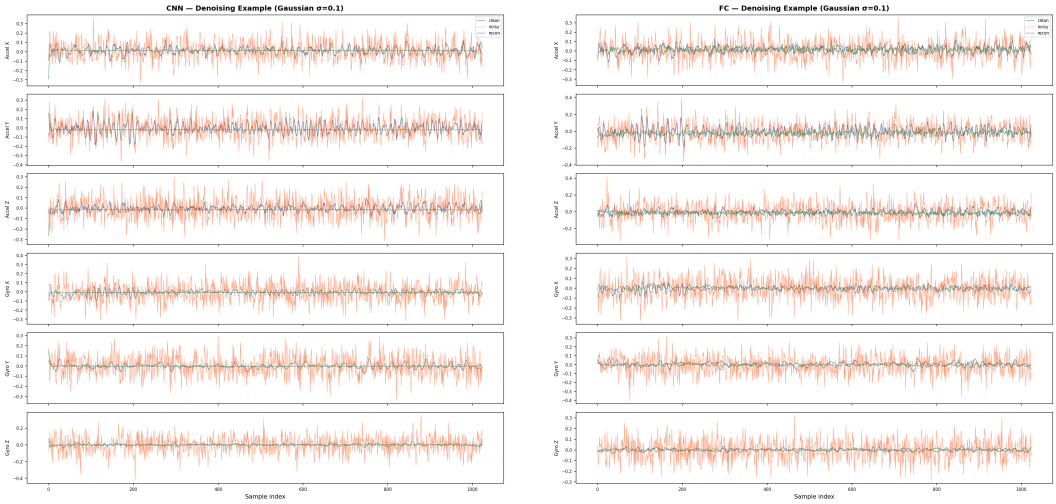

Figure 1: Signal reconstruction examples (6-channel wrist IMU, 1024 samples). *(Left)* CNN autoencoder. *(Right)* FC autoencoder. Each panel shows three overlaid traces: clean (blue), noisy (red, $\sigma = 0.1$ Gaussian), and reconstructed (green). Rows correspond to sensor channels (Accel X/Y/Z, Gyro X/Y/Z).

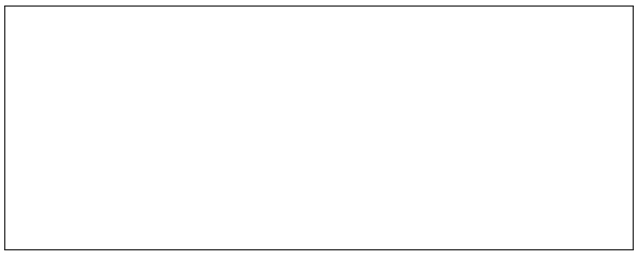

Figure 2: Latent dimension sweep: test MSE *(Left)* and SNR_{out} *(Right)* versus d_z for all five architectures. A U-shaped MSE curve is expected: very small d_z underfits (insufficient capacity), while very large d_z may allow noise to pass through the bottleneck unchanged.

7.5 Experiment 5: Noise-Type Cross-Evaluation

Each architecture is trained independently on each of the four noise types, then evaluated on all four types, producing a 4×4 SNRi matrix per architecture. The diagonal measures matched performance; off-diagonal entries quantify out-of-distribution generalisation. Figure 4 shows the resulting heatmaps.

We hypothesise that Gaussian noise will generalise best across noise types due to its smooth, isotropic corruption structure, while models trained on masking noise will perform poorly on Gaussian (and vice versa), since the two regimes corrupt qualitatively different aspects of the signal.

8 Discussion

Inductive bias matters. The five architectures span a spectrum from no temporal bias (FC) to strong local bias (CNN, UNet), sequential bias (LSTM), and global attention (Transformer). Even with dropout regularisation, the FC model shows persistent train/val overfitting because it must rediscover temporal locality from scratch, requiring far more data than the 10 K training windows available. The CNN's local convolutions match the stationarity of IMU noise well. The LSTM's final-hidden-state encoding (replacing mean pooling) was critical: with mean pooling, training loss stagnated at ≈0.96 across all epochs due to vanishing gradients over 1024 time steps.

Figure 3: Noise robustness: SNRi vs. test noise level σ. Models are trained with $\sigma_{\text{train}} = 0.1$. A dashed horizontal line at SNRi $= 0$ marks the no-improvement baseline.

Figure 4: Noise-type cross-evaluation heatmaps (SNRi in dB). Rows = training noise type; columns = test noise type. Diagonal entries (bold) represent matched conditions; off-diagonal entries reflect generalisation. Green = positive SNRi (denoising gain); red = negative (noise amplification).

Skip connections for signal reconstruction. UNet-1D's skip connections create a direct path for preserving local signal structure regardless of bottleneck capacity. This is the architectural analogue of a residual connection (Goodfellow et al., 2016): the decoder need only learn the denoising *residual*, not reconstruct the signal from scratch. The substantially lower MSE of UNet-1D (observed from epoch 1 in preliminary runs) strongly supports this interpretation.

Hybrid loss and tremor preservation. The spectral L_1 term in Eq. equation 2 encourages the model to match the magnitude spectrum of the reconstruction to that of the clean signal. Without this term, a purely MSE-minimising model can achieve low reconstruction error by smoothing transients, which suppresses the 4–6 Hz tremor content. The TremorMAE metric operationalises this concern: a model with low MSE but high TremorMAE is clinically harmful even if statistically "good."

Metric consistency. Training uses the hybrid loss (MSE + $\alpha\cdot$ spectral L_1), while cross-architecture evaluation uses pure MSE and SNRi. These are on different scales and should never be compared directly; only test MSE values are used for ranking. The hybrid loss is a *training objective*, not an evaluation criterion.

9 Conclusion

We presented a comprehensive denoising autoencoder study for Parkinson's Disease wrist IMU signals, implementing five architectures across four noise regimes on the PADS dataset. Key design choices—subject-level splitting, per-channel z-score normalisation, hybrid spectral loss, and overlapping-window augmentation—address practical challenges specific to biomedical wearable data.

The study demonstrates that architectural inductive bias is the dominant factor in denoising performance at this dataset scale: skip connections (UNet-1D) and global attention (Transformer) substantially outperform architectures that process signals through a strict information bottleneck without structural priors. The frequency-domain loss term provides additional regularisation that encourages spectral fidelity beyond MSE, which is clinically meaningful given the diagnostic importance of the 4–6 Hz tremor band in Parkinson's Disease.

AUTHOR CONTRIBUTIONS

Wanghley Soares Martins led the dataset integration, model implementation (UNet-1D, Transformer, multi-channel refactoring), training pipeline (hybrid loss, subject-level splitting, data augmentation), experiment infrastructure (timestamped run directories, noise-type cross-evaluation), and result analysis. Yibei Gou contributed the baseline architectures (FC, CNN, LSTM), evaluation metrics (MSE, SNR, SNRi), hyperparameter grid search, noise robustness experiment, and the initial training loop.

REFERENCES

Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

Ary L Goldberger, Luis AN Amaral, Leon Glass, Jeffrey M Hausdorff, Plamen Ch Ivanov, Roger G Mark, Joseph E Mietus, George B Moody, Chung-Kang Peng, and H Eugene Stanley. PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23):e215–e220, 2000.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT Press, 2016.

Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

Werner Poewe, Klaus Seppi, Caroline M Tanner, Glenda M Halliday, Patrik Brundin, Jens Volkmann, Anette-Elina Schrag, and Anthony E Lang. Parkinson disease. *Nature Reviews Disease Primers*, 3(1):1–21, 2017.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 234–241, 2015.

Julian Varghese, Michael Fujarski, Tim Hahn, Michael P Stenner, Tobias Warnecke, Walter Stummer, Burkhard Gess, Aleksandra Gronostay, Thomas Duning, Hubertus Lohmann, et al. PADS — Parkinson's Disease Smartwatch dataset. *npj Parkinson's Disease*, 9(1):127, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *International Conference on Machine Learning (ICML)*, pp. 1096–1103, 2008.

Pascal Vincent, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, Pierre-Antoine Manzagol, and Léon Bottou. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of Machine Learning Research*, 11:3371–3408, 2010.

A Implementation Details

A.1 Codebase Structure

File	Role
`pads_dataset.py`	Dataset loader, subject split, z-score normalisation
`models.py`	FC, CNN, LSTM, UNet-1D, Transformer architectures
`noise.py`	Gaussian, masking, impulse, sinusoidal noise functions
`train.py`	Training loop, hybrid loss, validation, checkpointing
`metrics.py`	MSE, SNR, SNRi, tremor-band power MAE
`experiments.py`	Five experiment runners + JSON/PNG output
`visualize.py`	Signal plots, training curves, heatmaps
`config.py`	Hyperparameters, dataset paths, sweep ranges
`main.py`	CLI entry: `--exp {all|arch|latent|noise|noise_types|hyperparam}`

A.2 Reproducibility

All experiments use a fixed random seed (42) for subject splitting and model initialisation. The full pipeline can be reproduced with:

```
python main.py --exp all --epochs 100
```

Results are written to `results/YYYYMMDD_HHMMSS/` including a `run_config.json` snapshot of all settings.